Software Engineering

Software Architecture

3 Ba INF
2026 – 2027

Kasper Engelen
kasper.engelen@uantwerpen.be

1 Practical

- Deadline: **Sunday, October 11, 2026, 22u00**
- Hand in via **Blackboard** as `GroupX_softwareArchitecture.zip`
- Extra material:
  - The customer requirements can be found on Blackboard (`customer_requirements.pdf`).
  - The various linked articles that explain the design patterns.
  - Other articles/books that explain design patterns.

2 Context

In this assignment you will analyse the customer requirements, in particular the non-functional requirements. You will use these requirements to evaluate a number of possible design patterns. Concretely, you will learn:

- how to interpret the **non-functional requirements**,
- how to quickly **read up** on different design patterns and compare them to the requirements,
- how to **consider** different design patterns in the **context of a given project**,
- how to list the **strengths and weaknesses** of a design pattern with respect to the requirements,
- how to make **Spike stories** in order to propose a few **prototypes** to a development team,
- how to formulate **Y-statements** as part of Architectural Decision Records,
- explore how **Generative AI** may support the above.

3 Report and evaluation

The evaluation will be **in-person**, where we will discuss your solutions together. You are free to balance the work across your team of 3 students. At the evaluation **every student is expected to understand and explain** the solution.

While making the assignments you should make use of the attached templates. When you have finished the assignments, you can hand them in via **Blackboard as a zip-file**. The templates are essential to ensure a **smooth evaluation** and to ensure that you do not **miss out on any grades!** During this session you will explore how Generative AI can support software engineering activities. Two persons in the group will perform the assignments with traditional means (Section 4.1 — Section 4.4). The third person will use a Generative AI tool to work in parallel (Section 4.5). You will switch these roles over the course of the semester. Afterwards you will compare the results and reflect upon the use of Generative AI (Section 4.6).

Note that for the exams you will not have access to generative AI so you should be able to perform the tasks without such support.

4 Assignment

For this assignment you will analyse the requirements document and consider a number of **design patterns**. Keep in mind any specified **non-functional requirements**, such as scalability, maintainability, reliability, etc. For each design pattern you will think about how to apply it to the project, and the **strengths and weaknesses** of the pattern in the context of the project. Make sure you answer these questions not in the general sense, but instead consider this **specific project** and the customer requirements!

4.1 Non-functional Requirements

There is a **strong link** between the software architecture and the non-functional requirements. In order to correctly evaluate different design patterns, we will first analyse the requirements document to **identify the non-functional requirements** specified by the client.

Assignment 1

Carefully read the requirements document. Mark in yellow the sentences in the requirements document that correspond to non-functional requirements. Beware the fact that the difference between a functional and a non-functional requirement is not always clear.
Afterwards, derive the list of non-functional requirements, listing them with explicit numbers (NF-xx) and name them according to the corresponding quality attribute to ease the traceability. You can find the standard list of quality attributes in Section 5.4.

4.2 Possible Architectures

Now that we have a good idea of the requirements, we will have to consider which **architectures** would satisfy these non-functional requirements. Pay special attention to **persistence** (i.e. databases) and the **technical and historical debt** it can incur. Since the difference between non-functional requirements and functionalities is not always clear, it might be that an architecture can also be linked to functional requirements.

Assignment 2

1. Read up on the following design patterns:
   - 3-layer architecture (Slides 15 — 16)
   - Blackboard-architecture (Slides 20 — 22)
   - Microservices or Monolith architecture (Explanation; Slides 31 — 33))
2. For two design patterns of your choice, answer the following questions, using the templates specified in Section 5:
   (a) How would this architecture be applied to the system? What would the different **components** of the architecture look like?
   (b) What would the **advantages** be, given the requirements? Refer to the functional requirements with UC-xx + the name of the use case. Refer to the non-functional requirements with NF-xx + the quality attribute.
   (c) What would the **disadvantages** be, given the requirements? Refer to the functional requirements with UC-xx + the name of the use case. Refer to the non-functional requirements with NF-xx + the quality attribute.

4.3 Spike Stories

In order to realise the project, we will have to decide which architecture we will use. Given that such a decision has a big impact and is difficult to correct later on, we will first build some **prototypes**. Spike stories can be used to propose such prototypes to the development team and to convince the team that it is worthwhile to build such prototypes.

Assignment 3

For the two design patterns you chose in Section 4.2 rewrite them as Spike stories. Make sure you use the templates below (Section 5.5). Specify exactly which use-case/user story you would use for the prototype. Then specify the component that would be prototyped and which insights you want to obtain from this prototype. List a number of conditions of satisfaction.

You can find some examples of Spike stories on Slide 42.

4.4 Y-statement

Assume that we delivered the spike stories and confirmed the advantages and disadvantages on our system. We now need to record the corresponding architecture decision using a Y-statement.

Assignment 4

For the two spikes stories you chose in Section 4.3 derive the corresponding Y-statement. Make sure you use the templates below (Section 5.6).

You can find some explanation on the Y-statement on Slide 46.

4.5 Same assignment using Generative AI

All previous assignments were performed by your team mates by hand. In this part, you will instead use Generative AI to perform a few of the tasks above.

We recommend that you make use of Microsoft Copilot, provided by the University, but you are also allowed to use another **paid** Generative AI tool such as Claude, ChatGPT, etc. Do not use AI tools that are free, since those are sub-optimal and do not reflect the tools you will use in your later job.

Assignment 5

Complete the following tasks:

- Ask the Generative AI to extract those sentences that correspond to **non-functional requirements** (Section 4.1). List them explicitly with numbers NF-XX and a reference to the quality attribute.
- Ask the Generative AI to evaluate the use of the design patterns in the system (Section 4.2). Best to provide a list of use cases as extracted in the session on requirements engineering.
  - Try to fool the generative AI by requesting design patterns which don't exist (i.e. Baroque, Rococo, ...) or referring to quality attributes which bear no meaning in a software engineering context (i.e. blood pressure, ...).
  - Some Generative AI tools will suggest a few follow-up tasks after each answer. Choose one of these.
- Ask the Generative AI to deduce the spike stories (Section 4.3) and the Y-statements (Section 4.4) .

For the report, first tell us who performed the manual part and who performed the part supported with Generative AI; you need to switch between these roles over the course of the semester. Then tell us which Generative AI tool you used and how long your chat session lasted. If possible also report how many tokens you used.

Then for each task, report the following:

- What prompt techniques (zero-shot, few-shot, chain-of-thought, ...) did you use? Provide the exact prompt(s) as well.

- Did you have to refine the prompt? If yes explain why this was needed and how you improved?
- How did you verify whether the response was good, acceptable, correct, useful . . . ?

4.6 Compare and Reflect

Compare the results from the manual assignments against the ones done with Generative AI support and reflect.

Assignment 6

- Where did the list of non-functional requirements (Section 4.1) differ?
- Where did the analysis of the use of design patterns (Section 4.2) differ?
- Which of the spike stories (Section 4.3) were easier to read? Were there conditions of satisfaction which were missing? Did the Generative AI come up with additional relevant conditions of satisfaction?
- Which of the Y-statements (Section 4.4) were easier to read?
- Did the Generative AI provide relevant insights, trade-offs? Of so, which?
- Did the Generative AI hallucinate?
- Where and how would you use Generative AI for Software Architecture? How does Generative AI complement the human-in-the-loop? Where would you **not** use it.

5 Templates

Below you will find a number of templates that you should use for the assignment. You can re-create these templates using a program of choice (LaTeX, MS Word, . . .).

5.1 Design pattern component explanations

Model View Controller	
Component	**Part of the project**
Controller	The steering wheel of the car
View	The indicators on the dashboard
. . .	. . .

5.2 Design pattern trade-off table

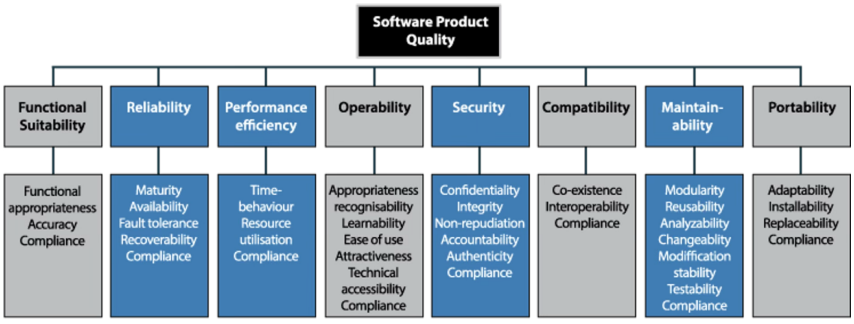
Visitor pattern	
Advantage	**Disadvantage**
Easy to implement and explain (NF-xx - understandability; NF-xx - maintaiability)	No support in Java (NF-xx - platform)
. . .	. . .

5.3 Link between the requirements and the design patterns

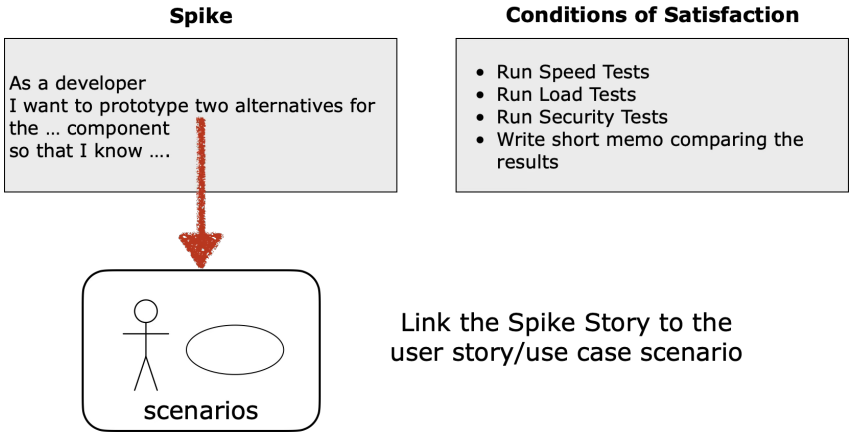
Aspect of the architecture	**Non-functional requirements**	**Functionality**
Singleton pattern	Existing MySQL database of the client (NF-xx - Reusability)	Easy-to-use customer database (UC-xx - Customer Registration)
Pipes and Filters	Re-usability of the components (NF-xx - Reusability)	Programming interface (UC-xx - Approve program enrolment)
. . .	. . .	. . .

5.4 Quality Attributes (Slide 10)

See also https://iso25000.com/en/iso-25000-standards/iso-25010



5.5 Spike story (Slide 42)



5.6 Y-statement (Slide 46)

The Y-statement reads as follows.

In the context of <use case UC-xx>, facing <non-functional concern c>, we decided for <option o1> and neglected <other options o2, o3, . . . >, to achieve <system qualities q1, q2, . . . /desired consequences>, accepting <downside/undesired consequences c>, because <additional rationale>.

